

深圳大学实验报告

课程名称： 算法设计与分析

实验名称： 实验五 桥

学院： 计算机与软件学院 专业： 软件工程

报告人： 黄瑜 学号： 2024150080 班级： 软件工程 01

同组人： 黄家成

指导教师： 吴定明

实验时间： 2026 年 6 月 14 日

实验报告提交时间：

本次实验代码：

[Kevin87654/algorithm-experiment-5-bridge: 算法分析与设计-实验 5-桥](#)

一. 实验目的

本实验主要研究无向图中的桥边问题。通过本实验，掌握图的连通性、连通分支、深度优先搜索以及并查集的基本原理和应用。

在无向图中，如果删除某一条边后，图中的连通分支数量增加，则称这条边为桥。等价地说，如果一条边不在任何环上，那么这条边就是桥。本实验需要实现基准算法，并设计一种使用并查集的高效算法，比较两种算法在不同规模图数据上的运行性能。

二. 实验内容与结果

1. 问题描述

给定一个无向图 G ，需要找出图中所有的桥边。桥边的判断依据是：删除该边后，如果图的连通分支数量增加，则该边为桥。

本实验使用的数据包括：

1. 图 2 示例图：共有 16 个顶点和 6 条桥，用于验证算法正确性。
2. mediumG.txt：中等规模无向图，用于测试基准算法和高效算法的性能。
3. largeG.txt：大规模无向图，用于测试高效算法在大数据集上的性能。

2. 数据结构设计

程序使用邻接表存储无向图。邻接表适合存储稀疏图，能够在较小空间内保存图结构，并且可以快速访问某个顶点的所有相邻顶点。

主要数据结构如下：

4. Edge：保存一条边的两个端点 u 和 v 。
5. AdjEdge：保存邻接点 to 和边编号 id 。
6. Graph：保存顶点数、边集合和邻接表。
7. DSU：并查集结构，支持 `find` 查找集合代表元素，并支持集合合并。
8. 在程序中，每条边都有唯一编号。这样在基准算法中可以根据边编号临时忽略某一条边，在高效算法中也可以根据边编号标记某条边是否为桥。

3. 基准算法设计

基准算法直接根据桥的定义进行判断。首先计算原图的连通分支数量，然后依次删除或忽略每一条边，并重新使用 DFS 计算删除该边后的连通分支数量。如果删除某条边后连通分支数量增加，则该边为桥。

基准算法伪代码：

```
输入：无向图  $G = (V, E)$ 
输出：所有桥边 bridges

bridges 初始化为空集合
baseComponents = 计算原图  $G$  的连通分支数量

对于每一条边  $e$  属于  $E$ ：
    临时忽略边  $e$ 
    currentComponents = 使用 DFS 计算当前图的连通分支数量
    如果 currentComponents > baseComponents：
        将  $e$  加入 bridges

返回 bridges
```

基准算法思路简单，能够直接体现桥的定义，适合用于小规模图和正确性验证。但是该算法需要对每一条边都重新进行一次图遍历，因此在大规模图上效率较低。

4. 高效算法设计

高效算法使用 DFS 生成森林，并结合并查集处理非树边。

首先对无向图进行 DFS，得到 DFS 树边和非树边。在无向图中，如果一条树边处在某个环上，则它不是桥；如果一条树边不属于任何环，则它是桥。非树边会连接 DFS 树中的两个顶点，从而形成一个环，因此该环路径上的所有树边都不是桥。

算法开始时，先把 DFS 树中的所有树边暂时标记为桥。然后遍历所有非树边，对于每一条非树边 (u, v) ，从 u 和 v 沿 DFS 树向上回溯，直到两者到达同一个祖先。在回溯过程中经过的树边都处于环中，因此将这些树边标记为非桥。为了避免重复回溯已经处理过的路径，使用并查集对已经处理过的路径进行压缩，从而提高算法效率。

高效算法伪代码：

```
输入：无向图  $G = (V, E)$ 
输出：所有桥边 bridges

对图  $G$  进行 DFS，生成 DFS 森林
对每一条 DFS 树边，先标记为桥
```

```
收集所有非树边
初始化并查集，每个顶点单独成集

对于每一条非树边 (u, v):
    u = Find(u)
    v = Find(v)

    当 u 和 v 不在同一个集合时:
        如果 depth[u] < depth[v]:
            交换 u 和 v

        将 u 到父节点的树边标记为非桥
        将 u 所在集合合并到 parent[u] 所在集合
        u = Find(u)

遍历所有边，仍然标记为桥的边即为最终桥边
返回 bridges
```

该算法利用并查集压缩已经处理过的 DFS 树路径，避免重复处理同一段路径，因此比基准算法更适合大规模图。

5. 复杂度分析

基准算法复杂度分析：

设图中顶点数为 V ，边数为 E 。使用邻接表存储图时，一次 DFS 遍历的时间复杂度为 $O(V+E)$ 。基准算法需要对每一条边都进行一次 DFS 判断，因此总时间复杂度为 $O(E*(V+E))$ 。

基准算法需要保存邻接表、访问标记数组和桥边结果，因此空间复杂度为 $O(V+E)$ 。

该算法实现简单，但当边数很大时，运行时间会急剧增加。例如 largeG.txt 中有 1000000 个顶点和 7586063 条边，如果对每条边都重新 DFS，计算量非常大，不适合实际运行。

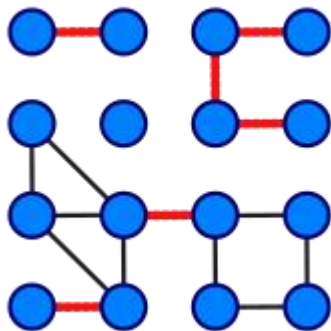
高效算法复杂度分析：

高效算法首先使用 DFS 遍历整张图，时间复杂度为 $O(V+E)$ 。之后遍历非树边，并使用并查集对已经处理过的路径进行压缩。由于已经压缩的路径不会被反复处理，因此整体运行效率远高于基准算法，运行时间接近 $O(V+E)$ 。

高效算法需要保存邻接表、DFS 父节点、深度、树边标记、桥边标记和并查集数组，因此空间复杂度为 $O(V+E)$ 。

6. 正确性验证

实验首先使用题目中图 2 的例子进行正确性验证。图 2 中共有 16 个顶点和 6 条桥。程序分别使用基准算法和高效算法进行计算，两个算法均得到 6 条桥，说明高效算法与基准算法结果一致。



程序输出的桥边为：

```
(0, 1)
(2, 3)
(2, 6)
(6, 7)
(9, 10)
(12, 13)
```

```
(base) PS D:\深圳大学\算法设计与分析\实验5> .\实验五.exe
===== Figure 2 correctness test =====
Vertices: 16, Edges: 6
Expected bridge count: 6
Baseline bridge count: 6, time: 0 ms
DSU bridge count: 6, time: 0 ms
Bridge edges found by DSU algorithm:
(0, 1)
(2, 3)
(2, 6)
(6, 7)
(9, 10)
(12, 13)
Result: PASS
```

运行结果显示，两个算法的桥数量均为 6，验证结果为 PASS，说明程序能够正确识别图 2 中的桥边。

截图位置建议：在本小节后插入“图 2 正确性验证运行结果截图”。

7. 实验结果

mediumG.txt 测试结果：

顶点数：250

```
边数: 1273
基准算法桥数量: 0
基准算法运行时间: 2 ms
高效算法桥数量: 0
高效算法运行时间: 0 ms
两个算法结果是否一致: YES
```

```
===== Dataset: mediumG.txt =====
Vertices: 250, Edges: 1273, load time: 0 ms
Baseline bridge count: 0, time: 2 ms
DSU bridge count: 0, time: 0 ms
Baseline and DSU results match: YES
```

largeG.txt 测试结果:

```
顶点数: 1000000
边数: 7586063
文件读取时间: 3708 ms
高效算法桥数量: 8
高效算法运行时间: 588 ms
```

```
===== Dataset: largeG.txt =====
Vertices: 1000000, Edges: 7586063, load time: 3708 ms
Baseline algorithm skipped for this dataset because it is O(E*(V+E)).
To force it, run this program with argument: --run-large-baseline
DSU bridge count: 8, time: 588 ms
```

对于 largeG.txt, 基准算法由于时间复杂度为 $O(E*(V+E))$, 在该规模数据上运行代价过高, 因此未进行完整运行。程序保留了强制运行基准算法的参数, 但默认跳过, 以避免长时间无结果。

8. 实验结果分析

从图 2 的验证结果可以看出, 基准算法和高效算法均找到了 6 条桥, 说明程序的桥边判断逻辑正确。

在 mediumG.txt 上, 两个算法得到的桥数量均为 0, 结果一致, 说明高效算法与基准算法在中等规模数据集上的输出一致。

在 largeG.txt 上, 高效算法能够在较短时间内完成计算, 得到桥数量为 8。由于该数据集中顶点数和边数都非常大, 基准算法需要对每一条边都重新遍历整张图, 运行代价过高, 因此不适合在该数据集上实际运行。

实验结果表明, 使用 DFS 树和并查集路径压缩的方法能够显著提高桥边查找效率, 尤其适合处理大规模无向图。

三. 实验总结

本实验实现了无向图中桥边查找的两种算法。基准算法直接根据桥的定义，通过删除边后判断连通分支数量是否增加来确定桥边，思路直观，但时间复杂度较高。高效算法利用 DFS 生成树和并查集，将处于环上的树边标记为非桥，从而避免了对每条边重复遍历整张图的问题。

通过图 2 的正确性验证，两个算法都找到了 6 条桥，结果一致。通过 `mediumG.txt` 和 `largeG.txt` 的性能测试可以看出，高效算法在大规模图数据上具有明显优势，能够在较短时间内完成桥边查找。

本实验加深了我对图的连通性、桥、DFS 遍历以及并查集应用场景的理解。尤其是在大规模图数据处理中，合理选择数据结构和算法设计方法对程序性能有非常明显的影响。

四. AI 使用报告

本次实验过程中，使用 Gemini 3 pro,GPT5.5 辅助完成了以下工作：

帮助阅读实验要求，梳理实验需要完成的内容。

写实验代码，使程序能够完成图 2 正确性验证、`mediumG.txt` 测试和 `largeG.txt` 测试。

辅助分析基准算法和高效算法的思路、伪代码和复杂度。

指导教师批阅意见：	
成绩评定：	
指导教师签字： 年 月 日	
备注：	

指导教师批阅意见：	
成绩评定：	
指导教师签字： 年 月 日	
备注：	

指导教师批阅意见：	
成绩评定：	
指导教师签字： 年 月 日	
备注：	

指导教师批阅意见：	
成绩评定：	
指导教师签字： 年 月 日	
备注：	

指导教师批阅意见：	
成绩评定：	
指导教师签字： 年 月 日	
备注：	

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。